FPGA-Based True Random Number Generation Using Programmable Delays in Oscillator-Rings

N. Nalla Anandakumar, Somitra Kumar Sanadhya, and Mohammad S. Hashmi, *Senior Member, IEEE*

***Abstract*—True random number generators (TRNGs) play a fundamental role in cryptographic systems. This brief presents a new and efficient method to generate true random numbers on field programmable gate array (FPGA) by utilizing the random jitter of free-running oscillators as a source of randomness. The free-running oscillator rings incorporate programmable delay lines (PDLs) to generate large variation of the oscillations and to introduce jitter in the generated ring oscillators clocks. The main advantage of the proposed TRNG utilizing PDLs is to reduce correlation between several equal length oscillator rings, and thus improve the randomness qualities. In addition, a Von Neumann corrector as post-processor is employed to remove any bias in the output bit sequence. The validation of the proposed approach is demonstrated on Xilinx Spartan-3A FPGAs. The proposed TRNG occupies 528 slices, achieves 6 Mb/s throughput with 0.999 per bit entropy rate, and passes all the national institute of standards and technology (NIST) statistical tests.**

***Index Terms*—TRNG, FPGA, ring oscillators, entropy, PDLs.**

I. Introduction

TRUE random number generators (TRNGs) are widely used in cryptographic applications such as key generation, random padding bits, and generation of challenges and nonces in authentication protocols. Moreover, TRNGs also find use in lottery drawings, computer games, gambling and probabilistic algorithms. The TRNGs must fulfill strict statistical requirements, be unpredictable and generate truly random numbers by making use of a physical source that is non-deterministic. In general, a poor random number generator often leads to decrease in the complexity of attacking a system using such a generator. For example, highly insecure pseudo-random number generator used on Mifare Classic tags reduced the attack complexity and allowed attackers access to the smart cards secret key [1]. These issues can be addressed by utilizing TRNGs to produce the needed random bits in cryptographic systems. The randomness of a TRNG is usually assessed through Diehard [2] and NIST [3] statistical tests while the unpredictability is verified by estimating entropy-per-bit through stochastic model [4].

Typical TRNGs use a single source of entropy and post-processing operation. Commonly used sources of entropy are thermal noise [5], metastability [6], [7], clock jitter in circuits [8], [9] and chaos [10]. Randomness is first extracted from the physical noise source and then this is interpreted into raw random bit-stream using a sampler (digitization). In practice, the raw random bit-stream usually does not bear good quality of randomness. Therefore, auxiliary post-processing operations, such as von Neumann corrector [11] or hash function [5] is required to improve the quality of the output TRNG bit stream and increase its randomness. In this context, a number of field programmable gate array (FPGAs) prototypes of TRNG with post-processing designs have been proposed [6], [7], [12], [13]. These designs derive entropy from the jitter of Ring Oscillators (RO) [12], [13] or the metastability of flip-flops [6], [7] which is caused by setup or hold time violations of flip-flops (FFs).

There are different problems that might arise in the construction of a TRNG based on oscillator rings implemented in FPGAs [14]. For example, the entropy of the output bit sequence from the TRNG would be drastically reduced when equal length oscillator rings configured in FPGAs are highly correlated with each other due to identical delays. To address this issue, we use programmable delay lines (PDLs) in the oscillator rings in the current work. This creates higher variation in RO oscillations from cycle to cycle and causes jitter in the generated RO clocks. Also, the output of the RO's are not correlated with each other due to the incorporation of the PDLs in the oscillator rings for each sampling clock which produces higher randomness. In addition, Von Neumann corrector is used as post-processor for improving statistical properties of the bitsteam produced by the proposed TRNG. We implement the base TRNG as well as the Von Neumann corrector on the same Xilinx Spartan-3A FPGAs (XC3S400A-4FTG256). The key contributions of this brief are:

- Proposal of an FPGA-based TRNG that uses PDL-induced random jitter in the clocks of free-running ROs as the source of randomness.
- Demonstration of effectiveness of the Von Neumann corrector as a post-processor for bias elimination.
- Experimental validation of the proposed TRNG and demonstration that it passes all tests in the NIST suite.
- The hardware evaluation results demonstrate high throughput-per-area and high entropy rate (i.e., true randomness) of the produced output bits.

The subsequent sections briefly discuss the Xilinx Spartan-3A FPGAs and PDLs, some existing RO based TRNGs, and

Manuscript received February 21, 2019; revised April 28, 2019; accepted May 21, 2019. Date of publication May 30, 2019; date of current version March 4, 2020. This brief was recommended by Associate Editor C. W. Sham. *(Corresponding author: N. Nalla Anandakumar.)*

N. N. Anandakumar is with the Hardware Security, Society for Electronic Transactions and Security, Chennai 600113, India, and also with the CSE Department, Indraprastha Institute of Information Technology Delhi, New Delhi 110020, India (e-mail: nallananth@gmail.com).

S. K. Sanadhya is with the Department of Computer Science and Engineering, Indian Institute of Technology Ropar, Rupnagar 140001, India (e-mail: somitra@iitrpr.ac.in).

M. S. Hashmi is with the School of Engineering, Nazarbayev University, Astana 010000, Kazakhstan, and also with the Department of Electronics and Communication Engineering, Indraprastha Institute of Information Technology Delhi, New Delhi 110020, India (e-mail: mshashmi@iiitd.ac.in).

Color versions of one or more of the figures in this paper are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TCSII.2019.2919891

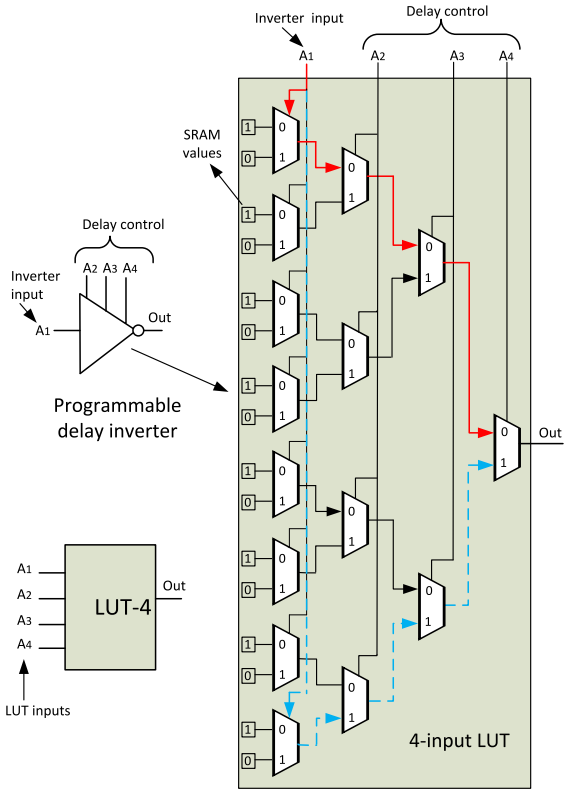

Fig. 1. PDL using a 4-input LUT.

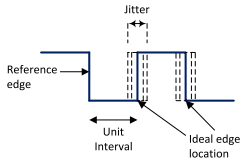

Fig. 2. Jitter in clock signals.

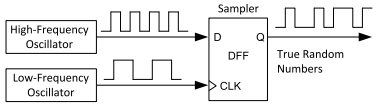

Fig. 3. Basic oscillator-based TRNG.

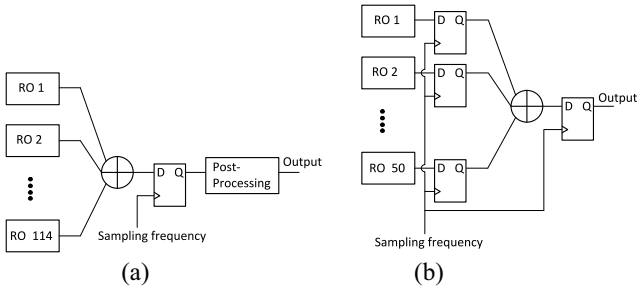

Fig. 4. Block diagram of the original TRNG (a) [12], and the modified TRNG (b) [15].

the details of the proposed TRNG. Subsequently experimental evaluation and comparisons in terms of area and throughput with existing techniques, and quality comparison by using the NIST statistical test suite are presented.

II. Preliminaries

A. Xilinx Spartan-3A FPGA Structure

For prototyping of the proposed TRNG, Spartan-3A FPGA from Xilinx is employed and it can be considered as a grid of interconnected Configurable Logic Blocks (CLBs) subdivided into four logical components called slices. It has two pairs of CLB slices, SLICEL and SLICEM, with each slice containing two 4-input Look-Up Tables (LUTs) and two flip-flops (FFs). In brief, SLICEL is the most basic type of slice and supports logic only while SLICEM supports both logic and memory functions (including RAM16 and SRL16 shift registers). The LUT based primitives are instantiated in hardware description language (HDL) as described in Spartan-3 Libraries Guide for HDL Designs.

B. Programmable Delay Lines (PDLs)

The internal variations of FPGA Look-Up Tables (LUT's) can be generated from changes in the LUT's propagation delays under different inputs [6]. For example, the LUT in Fig. 1 is programmed to implement an inverter whose LUT output (0) is always an inversion of its first input (A_1). Other inputs A_2, A_3, and A_4 act as "don't-care" bits but their values affect the signal propagation path from A_1 to the output (0). In this context, it has been shown, in Fig. 1, that the signal propagation path from A_1 to the output (0) is shortest for $A_2A_3A_4 = 000$ (marked with solid red line) and longest for $A_2A_3A_4 = 111$ (marked with dashed blue lines) for 4-input LUT's. Thus, a programmable delay inverter with three control inputs can be implemented by using one LUT. For the PDL, the first LUT input A_1 is the inverter input and the rest of the LUT inputs (three) are controlled by $2^3 = 8$ discrete levels.

III. Ring Oscillator Based TRNG

As mentioned earlier, a number of RO based TRNG designs have been reported in literature [12]–[18]. Typically, jitter is accumulated in the free-running RO's consisting of odd number of inverters or delay elements connected in ring configuration. This causes digital value of the oscillators output to change with a period of approximately $2DL$ where D is the delay of a single inverter and L is the number of inverters in an oscillator. Period of these oscillations vary from cycle to cycle causing jitter of the rising and falling edges of the generated RO clocks as shown in Fig. 2. Such oscillations, and hence jitter, in digital circuits can occur due to power supply variations, cross talks, semiconductor noise, temperature variations, and propagation delays. These jitters can be used to generate a stream of truly random bits using D flip-flop (DFF) based sampler for sampling the output of a high frequency oscillator as illustrated in Fig. 3.

The quality of generated true random bits can be improved by employing multiple free-running RO's [12]. This is achieved by feeding the outputs to an XOR tree (i.e., a multi-input XOR) and then sampling the XOR tree by a reference clock with a fixed frequency using a DFF to generate the random bit stream as shown in Fig. 4. However, it is very challenging for the XOR-tree and the sampling DFF to handle high number of switching activity from the free-running

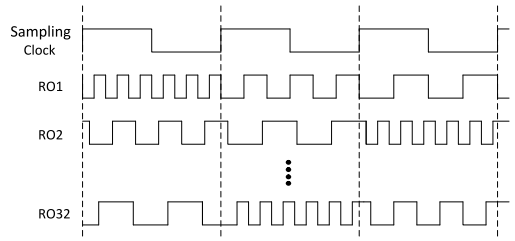

Fig. 5. RO outputs for each sampling clock by using PDL (variation in oscillations from CTC are not shown for clarity).

RO's in such designs [13]. It is due to the fact that high number of transitions during a sampling period due to parallel RO's place stringent setup and hold-time requirements. This aspect can be addressed to a small extent by incorporating a sampling DFF at the output of each free-running RO as shown in Fig. 4 [15]. This design passes the NIST statistical tests without post-processing and employs reduced number of RO's. However, it has similar mathematical complexity to the original design [12] and hence similar associated problems such as mutual dependence of rings [17] and correlation between the rings [14], which cause a lack of entropy at the output of the TRNG. The randomness qualities of the original TRNG [12] can also be improved at the cost of higher hardware resources [18] or a lower throughput [16].

IV. The Proposed TRNG Architecture

It is imperative to note that the RO based TRNGs, although exciting, are extremely limited in terms of randomness when identical RO's are employed [14]. Equal length oscillator rings configured in FPGA are highly correlated with each other due to identical delays and therefore the XOR of the output from these rings returns mostly zeros. This leads to poor randomness from the design. We show in this brief that the a TRNG incorporating PDL's can overcome this problem. Previously, the metastability of flip-flops was used for generating true random numbers [6]. They achieved metastability by using PDLs that accurately equalize the signal arrival times to flip-flops. On the other hand, this brief uses random jitter of free-running oscillators for generating true random numbers. We employ the PDL's in oscillator rings to generate large variation of the oscillations and to introduce jitter in the generated RO's clocks. The main advantage of the proposed TRNG utilizing PDL's is to reduce correlation between several equal length oscillator rings. For example, this can be achieved by variable RO outputs for each sampling clock by incorporating PDL as shown in Fig. 5. Moreover, the variation in RO oscillations from cycle to cycle (CTC) is also introduced by each oscillator ring due to inverter-delay. As a result, the XOR operation significantly improve the randomness qualities.

The implementation of the proposed TRNG along with the post-processing module on a low cost Xilinx Spartan-3A FPGAs is described in this section. Subsequently, Xilinx ISE design suite 13.4, the PyUSB-1.6 software, and the FT2232D USB interface is used for experimental evaluation of the proposed technique.

A. Design Overview

The proposed TRNG architecture is shown in Fig. 6. Here, each RO is realized using 3 inverters and 1 AND gate marked

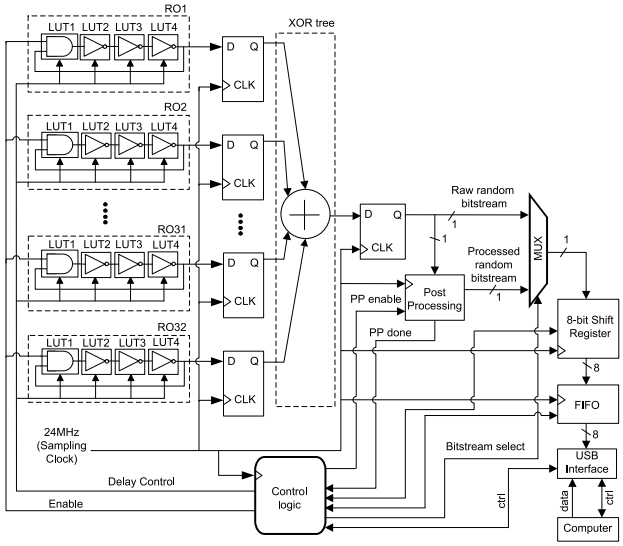

Fig. 6. Architecture of the proposed TRNG.

by black dashed boxes. The role of the AND gates is to enable the respective RO's. The design of the inverters and the AND gate require three and one LUT on the FPGA, respectively. In order to generate programmable delays inside the 4-input LUT, one of the LUT inputs is the ring connection while the other three inputs are configured with $2^3 = 8$ discrete levels. In case one uses 6-input LUT's (which are common in high-end FPGA's), one can use one input for ring connection and allow $2^5 = 32$ delay configurations for the remaining LUT inputs. This allows configuring 32 RO's without any constraints on the placement of the inverters.

The proposed architecture consists of 32 RO's, XOR tree, DFF's, shift register, FIFO (First-In, First-Out), and a post-processing unit. First, the control circuitry starts the 32 RO's simultaneously using the 'enable' input. The RO outputs are then combined by the XOR tree and sampled at the frequency clock of 24 MHz. If higher operating frequency is used for sampling then a frequency divider may be needed as well. Then, for the generation of PDLs, 2^3 discrete levels are arbitrarily applied to the delay control inputs for each sampling clock. Subsequently the sampled bits are either fed to the post-processor unit or directly sent to the FIFO without post-processing. Thus the output is either raw random bitstream or processed random bitstream selected via control input of the multiplexer and collected in blocks of 8 bits using the 8-bit shift register. Finally, each byte is stored in a FIFO of 64 byte width (i.e., 512 bits) and sent to PC through a USB interface for the TRNG statistical analysis. The FIFO allows reading of raw/processed random bitstream without flow interruption. The control logic module enables the start and stop of the RO's, FIFO, 8-bit shift register, post-processing unit, and selection of the raw/processed random bitstream for transfer to the PC.

B. Post-Processing

The proposed implementation employs a simple post-processing unit based on a von Neumann corrector [11] for enhancing the entropy and for removing any bias in the generated random bits. The post-processor also provides robustness of the TRNG output sequence. The employed Von Neumann corrector post-processing scheme is depicted in Fig. 7. We

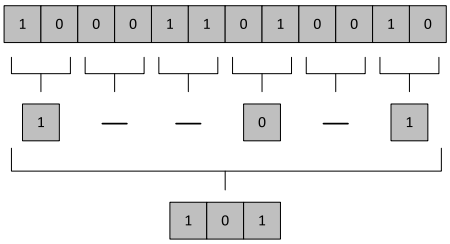

Fig. 7. Principle of operation of the Von Neumann.

TABLE I
COMPARISON OF THE PROPOSED METHOD WITH OTHER EXISTING TRNGS IMPLEMENTED ON XILINX FPGAS

TRNG Types	Area*			T.put (Mbps)	O.F. (MHz)	Platform
	LUTs	FFs	Slices			
THIS WORK	528	177	270	6	24	Spartan-3A (XC3S400A)
RO-based [18]	712	753	—	3.2	—	Spartan-3E (XC3S500E)
RO-based [19]	10	5	—	1.15	100	Spartan-6
RO-based [16]	521	131	—	2.57	—	Spartan-6
Meta stability [7]	—	—	580	12.5	100	Virtex-4 (XC4VFX20)
Meta stability [6]	128	—	—	2	—	Virtex-5 (XC3S500E)

* The Area for the TRNG with control logic (without USB interface).

read two bits at a time from the raw TRNG output and discard them if both of them are same (i.e., we eliminate 00 and 11 patterns). If the two bits are different (i.e., 01 or 10) then we take the first bit and discard the second bit. On an average, the post-processing unit requires 512 bits of raw input to generate 128 bits of post-processed output.

V. ANALYSIS AND DISCUSSION

A. Hardware Overhead Analysis

The results from the proposed TRNG architecture implemented on Xilinx Spartan-3A FPGAs are compared with recently reported TRNGs on Xilinx FPGAs in Table I. It is apparent that the proposed design outperforms earlier design [7] in terms of area. However, the throughput in our design is lower as we use a 24 MHz operating frequency (O.F) whereas the design in [7] uses a 100 MHz O.F. Furthermore, compared to previous works [6], [16], [19], our design exhibits higher throughput but occupies more area. The designs [6], [16], [19] employ more advanced FPGA configurations and hence achieves better area performance. On the other hand, our design outperforms [18] in terms of both the area and throughput although both are implemented on similar type of low cost FPGA. In summary, it can be concluded that the proposed TRNG architecture stands out as a potential candidate for lightweight secure applications considering that it provides very competitive area-throughput trade-offs.

B. Correlations Test

In order to investigate correlation between the rings used in the proposed TRNG, we computed the sample Pearsons correlation coefficient [14] between all combinations (i.e., $\binom{32}{2} = 496$) of the 32 RO's outputs. We generated 50,000 consecutive sampled bits from each of the RO's and used it for the test. The outputs of all these rings were sampled with an external sampling frequency of 24 MHz and 8 discrete levels

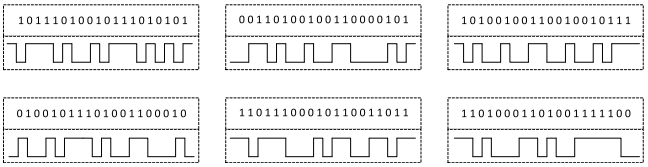

Fig. 8. 6 output bitstreams captured after restarting the TRNG.

were arbitrarily applied to the delay control inputs for each sampling clock. The correlation coefficient was observed to be ± 0.06 for all combinations. This allows us to conclude that the rings used in the proposed TRNG are not correlated with each other (i.e., correlation coefficient is close to 0). Moreover, we generated 80 million random bits from our proposed design and used it for the auto-correlation test specified in AIS-31 (T5) [4]. This test checks the bitwise correlation. The generated bit sequence passed the test and hence our proposed design has no correlation and dependency problems.

C. Measures of the Quality of Randomness

As per the standard practice in the domain, entropy test as the objective criterion of randomness is carried out. Followed by this, the restart test is carried out to provide evidence that the output, before post-processing, is distinct after restarting the system multiple times under the same conditions. Finally, the quality of the bitstream produced by the TRNG is tested using the NIST statistical test suite.

1) Entropy Estimation: The expected entropy per bit is 1 for an ideal true random number generator because the proportion of '0's and '1's is ideally 0.5. To estimate the entropy rate for the generated numbers, test $T8$ of the procedure B of the AIS-31 [4], available to test raw random numbers, is utilized in this brief. Considering a sequence of length N, test $T8$ splits the sequence in $Q + K$ disjoint L-bit words. The suggested parameters for the test $T8$ are $L = 8$, $Q = 2560$ and $K = 256000$. The minimum sequence of length required for this test is 7200000 bits. We generated 80 million random bits from our implementation and used it for the test. The bit sequence passed the test and achieved entropy of 7.9946 bits per byte (i.e., 0.9993 per bit). This is better than the minimum entropy requirement of 7.976 per byte (i.e., 0.997 per bit). Therefore, the proposed TRNG successfully fulfills the criteria for procedure B of the AIS-31 and can be used for applications that require high security. Moreover, Our proposed RO-based TRNG achieves higher entropy per bit when compared to earlier designs [16] (0.999 per bit) and [19] (0.997 per bit).

2) Restart Experiment: A test was carried out to verify the start up sequences for 6 restarts, from identical starting conditions, for the proposed design. First 20 sampled bits after every restart were observed and plotted as shown in Fig. 8. For a pseudorandom signal, the restart experiment produces similar graphs when the TRNG is repeatedly started from identical starting conditions. However, in our case, these were observed to be different each time and thus discard any pseudo randomness possibilities. Works [9], [13], and [15] used similar procedure to distinguish the amount of true randomness contained in a pseudorandom oscillating signal.

3) Statistical Evaluation of the Output: A set of randomness tests is standardized by NIST for evaluation of the quality of randomness of bitstreams [3]. The NIST statistical test suite is the most comprehensive publicly available tool. In essence,

TABLE II
NIST RANDOMNESS TEST RESULTS (UNIFORMITY OF P-VALUES AND PROPORTION OF PASSING SEQUENCE) FOR 1-GBITS RECORD THAT PASSED ALL TESTS (ROOM TEMPERATURE AT 1.2 V)

Statistical Test	Before Post-processing			After Post-processing		
	P-value	Prop.	Result	P-value	Prop.	Result
Frequency	0.5592	0.990	PASS	0.6199	0.992	PASS
Block-Freq.	0.6621	0.994	PASS	0.7441	0.991	PASS
Cumsum	0.3776	0.987	PASS	0.4138	0.987	PASS
Runs	0.4590	0.992	PASS	0.4276	0.993	PASS
Long-Run	0.7791	0.989	PASS	0.7011	0.990	PASS
Rank	0.7197	0.985	PASS	0.7634	0.988	PASS
FFT	0.5141	0.990	PASS	0.6593	0.994	PASS
Overlapping	0.0962	0.987	PASS	0.0805	0.991	PASS
Approx-Entropy	0.0553	0.988	PASS	0.2145	0.992	PASS
Serial	0.3345	0.991	PASS	0.4698	0.992	PASS
Linear-Complex	0.0088	0.989	PASS	0.2703	0.989	PASS
Nonperiodic	0.1573	0.990	PASS	0.4213	0.993	PASS
Universal	0.0909	0.982	PASS	0.2771	0.988	PASS
Rand-Excur	0.3394	0.986	PASS	0.3781	0.990	PASS
Rand-Variant	0.2587	0.984	PASS	0.3347	0.987	PASS

there are 15 type of tests which are typically carried out to assess the performance of TRNGs. In this brief, the parameters of each test were set as per the NIST recommendations [3]. The significance level α was chosen to be the default of 0.01 as it indicates 99% confidence interval. To examine the distribution of the P-values (randomness measure), 1000 times (sequences) of 10^6 bits both before and after post-processing were acquired. The statistical results of P-values and proportions (before and after post-processing) are given in Table II. If P-value is larger than 0.01 then the sequences are approximately uniformly distributed. Furthermore, a test is considered to have passed when the range of acceptable proportions are between 0.98056 and 0.99943 for α = 0.01 and sequences = 1000 [3]. The Proportion (Prop.) column indicates the number of P-values that were above the 0.01 confidence interval. The minimum proportion (minimum pass rate) for the tests is 0.982 before post-processing, and 0.987 after post-processing. The minimum pass-rate of sequences for our design (0.987) is higher in comparison to [9] (i.e., 0.97) and [7] (i.e., 0.986). The TRNG is then tested under two different temperature values of $55°C$ and $75°C$ using a temperature-controlled chamber while keeping the core supply voltage at 1.2 V. The minimum proportion for the tests is 0.981 at both these temperatures before post-processing. However, after post-processing, the minimum proportion becomes 0.983 and 0.982 at $55°C$ and $75°C$ respectively. In summary, it is evident from the results that the proposed TRNG exhibits better randomness (P-value is greater than 0.01 with greater proportion) properties at low hardware overhead and high throughput.

VI. CONCLUSION

A new design of RO-based TRNG is described and its implementation on Xilinx Spartan-3 FPGA is presented in this brief. The programmable delay of FPGA LUTs has been used to achieve random jitter and to enhance the randomness. It has been demonstrated that the proposed implementation provides a very good area-throughput trade-off. Effectively, it is capable of producing a throughput of 6 Mbps after post-processing with a low hardware footprint. In addition, the restart experiments show that the output of the proposed TRNG behaves truly random. It achieves high entropy rate and successfully passes all NIST statistical tests. It can actually be inferred that the proposed design has the potential to be a good candidate for lightweight security applications.

ACKNOWLEDGMENT

The authors would like to thank the anonymous reviewers for their helpful and constructive comments that greatly contributed in improving the final version of the article. The first author N. Nalla Anandakumar wish to thank Dr. N. Sarat Chandra Babu, Executive Director, SETS, Chennai for his heartfelt support.

REFERENCES

[1] K. Nohl, D. Evans, S. Starbug, and H. Plötz, "Reverse-engineering a cryptographic RFID tag," in *Proc. 17th Conf. Security Symp.*, 2008, pp. 185–193.
[2] G. Marsaglia, "Diehard: A battery of tests of randomness," 1996.
[3] L. E. Bassham, III *et al.*, "A statistical test suite for random and pseudo-random number generators for cryptographic applications, Rev. 1a," U.S. Dept. Commerce, Nat. Inst. Stand. Technol., Rep. SP 800–22, 2010.
[4] W. Schindler and W. Killmann, "A proposal for: Functionality classes for random number generators," 2011. [Online]. Available: https://www.bsi.bund.de/DE/Themen/ZertifizierungundAnerkennung/Produktzertifizierung/ZertifizierungnachCC/Anwendungshinweiseund Interpretationen/anwendungshinweiseundinterpretationen_node.html
[5] B. Jun and P. Kocher, *The Intel Random Number Generator*, Cryptography Res. Inc., San Francisco, CA, USA, Apr. 1999.
[6] M. Majzoobi, F. Koushanfar, and S. Devadas, "FPGA-based true random number generation using circuit metastability with adaptive feedback control," in *Proc. Cryptograph. Hardw. Embedded Syst. (CHES)*, 2011, pp. 17–32.
[7] H. Hata and S. Ichikawa, "FPGA implementation of metastability-based true random number generator," *IEICE Trans. Inf. Syst.*, vol. E95.D, no. 2, pp. 426–436, 2012.
[8] A. P. Johnson, R. S. Chakraborty, and D. Mukhopadyay, "An improved DCM-based tunable true random number generator for Xilinx FPGA," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 64, no. 4, pp. 452–456, Apr. 2017.
[9] D. Liu, Z. Liu, L. Li, and X. Zou, "A low-cost low-power ring oscillator-based truly random number generator for encryption on smart cards," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 63, no. 6, pp. 608–612, Jun. 2016.
[10] A. Beirami and H. Nejati, "A framework for investigating the performance of chaotic-map truly random number generators," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 60, no. 7, pp. 446–450, Jul. 2013.
[11] J. von Neumann, "Various techniques used in connection with random digits," in *Monte Carlo Method*. Washington, DC, USA: Nat. Bureau Stand. Appl. Math., Dec. 1951, pp. 36–38.
[12] B. Sunar, W. J. Martin, and D. R. Stinson, "A provably secure true random number generator with built-in tolerance to active attacks," *IEEE Trans. Comput.*, vol. 56, no. 1, pp. 109–119, Jan. 2007.
[13] M. Dichtl and J. D. Golić, "High-speed true random number generation with logic gates only," in *Proc. Cryptograph. Hardw. Embedded Syst. (CHES)*, 2007, pp. 45–62.
[14] K. Wold and S. Petrovivć, "Security properties of oscillator rings in true random number generators," in *Proc. IEEE 15th Int. Symp. Design Diagn. Electron. Circuits Syst. (DDECS)*, Apr. 2012, pp. 145–150.
[15] K. Wold and C. H. Tan, "Analysis and enhancement of random number generator in FPGA based on oscillator rings," in *Proc. Int. Conf. Reconfig. Comput. FPGAs*, Dec. 2008, pp. 385–390.
[16] O. Petura, U. Mureddu, N. Bochard, V. Fischer, and L. Bossuet, "A survey of AIS-20/31 compliant TRNG cores suitable for FPGA devices," in *Proc. 26th Int. Conf. Field Program. Logic Appl. (FPL)*, Aug. 2016, pp. 1–10.
[17] N. Bochard, F. Bernard, V. Fischer, and B. Valtchanov, "True randomness and pseudo-randomness in ring oscillator-based true random number generators," *Int. J. Reconfig. Comput.*, vol. 2010, Dec. 2010, Art. no. 879281.
[18] A. Maiti, R. Nagesh, A. Reddy, and P. Schaumont, "Physical unclonable function and true random number generator: A compact and scalable implementation," in *Proc. 19th ACM Great Lakes Symp. VLSI (GLSVLSI)*, 2009, pp. 425–428.
[19] B. Yang, V. Rožic, M. Grujic, N. Mentens, and I. Verbauwhede, "ES-TRNG: A high-throughput, low-area true random number generator based on edge sampling," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2018, no. 3, pp. 267–292, Aug. 2018.